

AgroSamridhi Platform

Step-by-step guide: containerizing the full microservices stack with Docker Compose

This is a planning guide only — no files have been changed. It documents exactly what to create and where, so the changes can be made deliberately and reviewed one at a time.

Overview

8 containers total: `mysql` and `redis` (already defined in `docker-compose.yml`), plus 6 new Spring Boot service images and 1 frontend image — all on one Docker network so containers can resolve each other by container name instead of `localhost`.

Step 1 — Add a `.dockerignore` file

Create one at the **repo root** to keep build contexts small and avoid copying stale build artifacts into images:

```
target/  
frontend/node_modules/  
frontend/dist/  
.git/  
*.log  
logs/
```

Step 2 — Dockerfile for each Spring Boot service

Same template for all six services: `discovery-server`, `api-gateway`, `auth-service`, `data-ingestion-service`, `ai-inference-service`, `dashboard-aggregator-service`. Create a `Dockerfile` **inside each service folder**, e.g. `auth-service/Dockerfile`:

```
# ---- build stage ----
FROM maven:3.9-eclipse-temurin-17 AS build
WORKDIR /workspace

# copy the whole multi-module reactor so `-am` can resolve the parent + sibling deps
COPY pom.xml .
COPY discovery-server/pom.xml discovery-server/pom.xml
COPY api-gateway/pom.xml api-gateway/pom.xml
COPY auth-service/pom.xml auth-service/pom.xml
COPY data-ingestion-service/pom.xml data-ingestion-service/pom.xml
COPY ai-inference-service/pom.xml ai-inference-service/pom.xml
COPY dashboard-aggregator-service/pom.xml dashboard-aggregator-service/pom.xml
RUN mvn -q -pl auth-service -am dependency:go-offline

COPY discovery-server discovery-server
COPY api-gateway api-gateway
COPY auth-service auth-service
COPY data-ingestion-service data-ingestion-service
COPY ai-inference-service ai-inference-service
COPY dashboard-aggregator-service dashboard-aggregator-service
RUN mvn -q -pl auth-service -am package -DskipTests

# ---- runtime stage ----
FROM eclipse-temurin:17-jre
WORKDIR /app
COPY --from=build /workspace/auth-service/target/*.jar app.jar
EXPOSE 8081
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Change `-pl auth-service` (both occurrences) and the `COPY --from=build` jar path / `EXPOSE` port for each service's own Dockerfile:

Service	Port
discovery-server	8761
api-gateway	8080
auth-service	8081
data-ingestion-service	8083
ai-inference-service	8084
dashboard-aggregator-service	8090

Because these Dockerfiles reference the parent `pom.xml` and sibling modules, they must be built with the **repo root as build context**, not the service folder — set in `docker-compose.yml` (Step 6), not on the command line.

Step 3 — Frontend Dockerfile

Create `frontend/Dockerfile` :

```
# ---- build stage ----
FROM node:22-alpine AS build
WORKDIR /app
COPY frontend/package*.json ./
RUN npm ci
COPY frontend .
ARG VITE_API_BASE_URL=http://localhost:8080
ENV VITE_API_BASE_URL=$VITE_API_BASE_URL
RUN npm run build

# ---- runtime stage ----
FROM nginx:alpine
COPY --from=build /app/dist /usr/share/nginx/html
COPY frontend/nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
```

Create `frontend/nginx.conf` — **this is required**. Without it, React Router's client-side routes (`/weather` , `/mandi` , etc.) will 404 on refresh because Nginx tries to find a matching file instead of falling back to `index.html` :

```
server {
    listen 80;
    location / {
        root /usr/share/nginx/html;
        index index.html;
        try_files $uri $uri/ /index.html;
    }
}
```

Key gotcha: Vite bakes `VITE_API_BASE_URL` into the JS bundle at **build time**, not runtime. Since the browser (not another container) calls the gateway, `http://localhost:8080` still works correctly even inside Docker — as long as api-gateway's port is mapped to `8080` on the host. The default `ARG` above is fine unless deploying somewhere the gateway isn't reachable at `localhost:8080` from the browser.

Step 4 — The "localhost" problem (config overrides, no file edits needed)

Every service's `application.properties` / `.yaml` hardcodes `localhost` for Eureka, MySQL, and Redis. Inside Docker, containers must address each other by **service name**, not `localhost`. These files do not need to be edited — Spring Boot's relaxed env-var binding lets any property be overridden from `docker-compose.yml`'s `environment:` block. Exactly what to override, per service:

discovery-server

No overrides needed — it's the Eureka server itself.

api-gateway

```
EUREKA_CLIENT_SERVICEURL_DEFAULTZONE: http://discovery-server:8761/eureka/
```

auth-service

```
EUREKA_CLIENT_SERVICEURL_DEFAULTZONE: http://discovery-server:8761/eureka/  
SPRING_DATASOURCE_URL: jdbc:mysql://agrosamridhi-db:3306/agrosamridhi_db?allowPublicKeyRetrieval=true  
SPRING_DATASOURCE_PASSWORD: rootpassword
```

data-ingestion-service

```
EUREKA_CLIENT_SERVICEURL_DEFAULTZONE: http://discovery-server:8761/eureka/  
SPRING_DATASOURCE_URL: jdbc:mysql://agrosamridhi-db:3306/agrosamridhi_db?createDatabaseIfNotExist=true  
SPRING_DATASOURCE_PASSWORD: rootpassword
```

ai-inference-service

```
EUREKA_CLIENT_SERVICEURL_DEFAULTZONE: http://discovery-server:8761/eureka/
```

dashboard-aggregator-service

```
EUREKA_CLIENT_SERVICEURL_DEFAULTZONE: http://discovery-server:8761/eureka/  
SPRING_DATA_REDIS_HOST: agrosamridhi-cache
```

The one thing that genuinely needs a file edit (env vars can't reach it — it's a `WebMvcConfigurer` list, not a property): `api-gateway/src/main/java/.../config/CorsConfig.java` already allows `http://localhost:3000`, and since the **browser** sends the Origin header (not a container), this keeps working unchanged as long as the frontend container is mapped to host port `3000`. No action needed.

Also not fixable via env var: `api-gateway/src/main/resources/application.properties` hardcodes each route's target as `http://localhost:8081`, `:8083`, `:8084`, `:8090`. These need to become `http://auth-service:8081`, `http://data-ingestion-service:8083`, etc. — either as a real file edit, or via indexed env vars like `SPRING_CLOUD_GATEWAY_MVC_ROUTES_0_URI=http://auth-service:8081` (verify locally; indexed properties as env vars are less common than simple keys).

Step 5 — Move secrets to a `.env` file

The MySQL password (`rootpassword`), the JWT secret, and the Gemini/Agmarknet API keys are currently hardcoded across various config files. Since `docker-compose.yml` is being touched anyway, this is the natural point to stop committing secrets.

Create `.env` at the repo root (and confirm it's excluded in `.gitignore`):

```
MYSQL_ROOT_PASSWORD=rootpassword
JWT_SECRET=agro_auth_super_secret_key_minimum_256_bits_long_abc123xyz456
GEMINI_API_KEY=<your-key>
AGMARKNET_API_KEY=<your-key>
```

`docker-compose.yml` auto-loads `.env` ; reference values as `${MYSQL_ROOT_PASSWORD}` in `environment:` blocks. `app.jwt.secret` in `auth-service/api-gateway` is already written as `${JWT_SECRET:default}` , so it already respects an env var — just inject `JWT_SECRET` for both containers. `gemini.api.key` and `agmarknet.api-key` are **not** currently written with `${...}` placeholders, so overriding those via env var needs a small `${GEMINI_API_KEY}` / `${AGMARKNET_API_KEY}` placeholder edit in their config files first.

Step 6 — Extend `docker-compose.yml`

Add to the existing file (which already has `agrosamridhi-db` and `agrosamridhi-cache`). Shape for each new service:

```

discovery-server:
  build:
    context: .
    dockerfile: discovery-server/Dockerfile
  ports:
    - "8761:8761"

api-gateway:
  build:
    context: .
    dockerfile: api-gateway/Dockerfile
  ports:
    - "8080:8080"
  environment:
    EUREKA_CLIENT_SERVICEURL_DEFAULTZONE: http://discovery-server:8761/eureka/
  depends_on:
    - discovery-server

auth-service:
  build:
    context: .
    dockerfile: auth-service/Dockerfile
  ports:
    - "8081:8081"
  environment:
    EUREKA_CLIENT_SERVICEURL_DEFAULTZONE: http://discovery-server:8761/eureka/
    SPRING_DATASOURCE_URL: jdbc:mysql://agrosamridhi-db:3306/agrosamridhi_db?allowPublicKeyRetrieval=true
    SPRING_DATASOURCE_PASSWORD: ${MYSQL_ROOT_PASSWORD}
  depends_on:
    agrosamridhi-db:
      condition: service_healthy
    discovery-server:
      condition: service_started

frontend:
  build:
    context: .
    dockerfile: frontend/Dockerfile
  args:
    VITE_API_BASE_URL: http://localhost:8080
  ports:
    - "3000:80"
  depends_on:
    - api-gateway

```

Repeat the pattern (with the env vars from Step 4) for `data-ingestion-service`, `ai-inference-service`, `dashboard-aggregator-service`. Note `context: .` (repo root) with `dockerfile:` pointing into the subfolder — this is what makes the multi-module Maven build work.

`agrosamridhi-db` already has a `healthcheck:` (mysqladmin ping) — that's what `condition: service_healthy` waits on. Consider adding one to `agrosamridhi-cache` too (`redis-cli ping`).

On healthchecks for the Spring services: only `auth-service` currently has `spring-boot-starter-actuator` on its classpath — the other 5 don't. Without it there's no `/actuator/health` to poll. Either add the actuator dependency to the other services' `pom.xml` s for proper `depends_on: condition: service_healthy` chains, or accept that those services will simply retry their Eureka registration until dependencies are ready — which is what happens today anyway, and is fine for local dev.

Step 7 — Build and run

```
docker compose up --build
```

First build will be slow (full Maven downloads per service unless layers are shared/cached). Subsequent builds are much faster via Docker layer caching, as long as `pom.xml` files don't change.

Step 8 — Verify

Same checks as the manual testing pass, now against containers:

```
docker compose ps
curl http://localhost:8761/eureka/apps # all 5 services registered
curl -X POST http://localhost:8080/api/auth/login -d '...' # through the gateway
```

Open `http://localhost:3000` — should behave identically to the current `npm run dev` setup.

One more consideration before starting

MySQL data lives in the `db_data` named volume. If MySQL has been running as a bare container (not via this compose file) with different init state, double-check `db_data` already reflects the schema fixes applied manually (the `caste_category` column width, the `primaryCrop` column) — those were applied directly against the running container, so they're already baked into that volume and will persist across `docker compose up` as long as the volume isn't deleted.